

Enterprise-Level Scenario-Based MCQs (Python Back-end, APIs, Testing, DevOps)

1. Flask Architecture & Request Handling

A fintech company is building an internal reimbursement portal using Flask. Employees submit reimbursement requests through a web form, and managers review requests on a dashboard. During testing, developers notice that every form submission reloads the page but no data is saved to the database.

Which issue is MOST likely causing the problem?

- A. The Flask application is missing template inheritance
- B. The route only accepts GET requests instead of POST requests**
- C. Jinja2 templates are not installed
- D. Flask cannot process HTML forms directly

Answer:

B. The route only accepts GET requests instead of POST requests

Explanation:

HTML forms usually submit data using POST. If the Flask route does not include `methods=['POST']`, form data will not be processed correctly.

2. Flask Templates & Responses

An HR portal built using Flask displays employee salary reports. The backend successfully fetches salary data, but the UI page shows blank values even though no errors appear in logs.

What is the MOST probable cause?

- A. The Flask server is not running in debug mode
- B. Variables were not passed properly to the Jinja template**
- C. HTML cannot display Python objects
- D. Flask requires JavaScript for template rendering

Answer:

B. Variables were not passed properly to the Jinja template

Explanation:

Jinja templates require explicit variable passing through `render_template()`. Missing variables result in empty placeholders.

3. FastAPI Request Validation

A healthcare platform exposes an API endpoint for patient registration using FastAPI and Pydantic models. During testing, invalid age values like `"twenty"` are rejected automatically.

Which FastAPI feature is responsible for this behavior?

- A. SQLAlchemy validation
- B. Flask-WTF forms
- C. Pydantic request validation**
- D. Uvicorn middleware

Answer:

C. Pydantic request validation

Explanation:

FastAPI uses Pydantic models for automatic request validation and type enforcement.

4. FastAPI Path & Query Parameters

A logistics company provides an endpoint:

`/shipment/{shipment_id}?status=delivered`

What is `shipment_id` considered in FastAPI?

- A. Header parameter
- B. Query parameter
- C. Path parameter
- D. Body parameter

Answer:

C. Path parameter

Explanation:

Values embedded directly in the URL path are path parameters.

5. REST API Principles

An e-commerce platform redesigns its APIs to align with REST principles. Developers are reviewing endpoint naming standards.

Which endpoint BEST follows REST conventions for retrieving all products?

- A. */getAllProducts*
- B. */products/list*
- C. */products*
- D. */fetchProductsNow*

Answer:

C. */products*

Explanation:

REST APIs use nouns rather than actions. HTTP methods define operations.

6. CRUD Operations

A banking application exposes the following API operations:

- Create account
- Retrieve account details
- Update customer address
- Remove account

Which HTTP method should be used for removing an account?

- A. GET
- B. POST
- C. PUT
- D. DELETE**

Answer:

D. DELETE

Explanation:

DELETE is the standard HTTP method for removing resources.

7. API Best Practices

A retail organization's APIs are failing because clients cannot understand server errors. The API team decides to standardize response structures.

Which practice is MOST appropriate?

- A. Returning plain text errors
- B. Returning random numeric codes
- C. Returning structured JSON responses with HTTP status codes**
- D. Returning HTML pages for failures

Answer:

C. Returning structured JSON responses with HTTP status codes

Explanation:

Well-designed APIs provide structured responses with meaningful HTTP status codes.

8. Postman API Testing

A QA engineer wants to automatically verify whether an API response status is *200 OK* in Postman.

Which Postman feature should be used?

- A. Collections
- B. Monitors
- C. Tests tab with assertions
- D. Environment variables

Answer:

C. Tests tab with assertions

Explanation:

The Tests tab allows automated validation using JavaScript assertions.

9. Git Basics

A developer accidentally commits buggy code directly to the main branch of a production repository.

Which Git best practice would have prevented this issue?

- A. Using force push
- B. Working directly on production branch
- C. Using feature branches and pull requests
- D. Deleting local repositories regularly

Answer:

C. Using feature branches and pull requests

Explanation:

Feature branches isolate development work and enable peer reviews before merging.

10. Virtual Environments

Two Python projects on the same system require different versions of FastAPI. Developers face dependency conflicts.

What is the BEST solution?

- A. Reinstall Python globally
- B. Use separate virtual environments**
- C. Delete pip cache
- D. Use one shared requirements file

Answer:

B. Use separate virtual environments

Explanation:

Virtual environments isolate dependencies between projects.

11. Pytest Fixtures

A telecom company's automated tests repeatedly create database connections before every test case, increasing execution time significantly.

Which Pytest feature can optimize this?

- A. Flask blueprints
- B. Fixtures**
- C. Path parameters
- D. Middleware

Answer:

B. Fixtures

Explanation:

Fixtures help reuse setup logic efficiently across tests.

12. Assertions in Testing

A developer writes this test:

```
assert response.status_code == 200
```

What is the PRIMARY purpose of this assertion?

- A. To deploy the application
- B. To validate expected behavior
- C. To improve API speed
- D. To create database entries

Answer:

B. To validate expected behavior

Explanation:

Assertions confirm whether actual outputs match expected results.

13. CI/CD Pipelines

A company wants every GitHub push to automatically trigger testing and deployment validation.

Which concept supports this requirement?

- A. Manual deployment
- B. CI/CD pipeline
- C. Flask routing
- D. Database indexing

Answer:

B. CI/CD pipeline

Explanation:

CI/CD automates testing, integration, and deployment workflows.

14. GitHub Actions

A DevOps engineer creates a workflow file under `.github/workflows/`.

What is the PRIMARY purpose of this file?

- A. Database backup
- B. Docker image cleanup
- C. Defining automated CI/CD workflows
- D. API authentication

Answer:

C. Defining automated CI/CD workflows

Explanation:

GitHub Actions workflows automate tasks such as testing and deployment.

15. Docker Containerization

A machine learning application works on the developer's laptop but fails on production servers due to missing dependencies.

How does Docker help solve this issue?

- A. By replacing Python completely
- B. By packaging application code and dependencies together
- C. By removing the operating system
- D. By converting APIs into databases

Answer:

B. By packaging application code and dependencies together

Explanation:

Docker containers ensure consistent runtime environments across systems.

16. Docker Images vs Containers

A junior developer is confused between Docker images and containers.

Which statement is CORRECT?

- A. Images are running instances of applications
- B. Containers are templates used to create images
- C. Images are blueprints, while containers are running instances
- D. Containers cannot run Python applications

Answer:

C. Images are blueprints, while containers are running instances

Explanation:

Docker images define application environments; containers are executable instances.

17. SQL CRUD Operations

An insurance company database stores policyholder information. A developer wants to update a customer's phone number.

Which SQL operation should be used?

- A. INSERT
- B. SELECT
- C. UPDATE
- D. DELETE

Answer:

C. UPDATE

Explanation:

UPDATE modifies existing records in a table.

18. ORM Concepts with SQLAlchemy

A software architect recommends using SQLAlchemy ORM instead of raw SQL queries.

What is the MAIN advantage?

- A. It eliminates databases entirely
- B. It allows interaction with databases using Python objects
- C. It converts APIs into Docker containers
- D. It removes the need for testing

Answer:

B. It allows interaction with databases using Python objects

Explanation:

ORMs simplify database operations using object-oriented programming concepts.

19. MongoDB & NoSQL

A social media company stores rapidly changing user activity logs with flexible schema requirements.

Why is MongoDB a suitable choice?

- A. It only supports structured tabular data
- B. It is optimized for relational joins
- C. It supports flexible document-based storage
- D. It cannot handle JSON data

Answer:

C. It supports flexible document-based storage

Explanation:

MongoDB stores data as flexible JSON-like documents, ideal for dynamic schemas.

20. CSV & JSON Data Integration

An analytics team receives customer data from multiple vendors in CSV and JSON formats. Before importing into databases, the data must be validated and transformed.

Which activity BEST describes this process?

- A. API deployment
- B. Data integration and preprocessing**
- C. Container orchestration
- D. Route handling

Answer:

B. Data integration and preprocessing

Explanation:

CSV/JSON parsing and validation are part of data integration workflows used before storage or analytics.